BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI

Semester: 5

Academic Year: 2026-27

Practical No: 4

Division: C Batch: 2

Roll no: 3090341 PRN No: 12410595

Name of Student: Pranav Khatod

Subject Name : Deep Learning

Problem Statement:

Implement a Simple RNN for sequence classification using the IMDB movie review dataset. Preprocess variable-length reviews, train an embedding + SimpleRNN model, and evaluate accuracy, precision, recall, F1-score, and the confusion matrix.

Algorithm / Methodology:

Libraries Used:

1. TensorFlow / Keras (imdb, pad_sequences, Sequential, Embedding, SimpleRNN, Dense)
2. NumPy
3. Matplotlib
4. scikit-learn metrics

Data Preparation

The IMDB dataset is loaded with a vocabulary size of 10,000 most frequent words. Reviews are padded or truncated to a maximum length of 200 tokens using pad_sequences. Both training and test sets contain 25,000 samples of shape (25000, 200).

Model Architecture

Embedding Layer: Maps word indices to 64-dimensional vectors (vocab_size=10000, input length=200).

SimpleRNN Layer: 64 recurrent units to model sequential dependencies in the review.

Output Dense Layer: 1 neuron with Sigmoid activation for binary sentiment classification.

Model Compilation and Training

The model is compiled with Adam optimizer and binary_crossentropy loss, tracking accuracy. It is trained for 3 epochs with batch size 128 and a validation split of 0.2.

Experiments and Results

Validation accuracy improved from 0.6560 (epoch 1) to 0.7456 (epoch 2) and 0.8098 (epoch 3). On the test set the model achieved accuracy 0.8144, precision 0.8004, recall 0.8375, and F1-score 0.8186.

Confusion matrix on the test set: [[9890, 2610], [2031, 10469]]. A sample interactive prediction is also demonstrated using a typed review.

Code :

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix,
classification_report

vocab_size=10000
max_length=200
(x_train,y_train),(x_test,y_test)=imdb.load_data(num_words=vocab_size)
x_train=pad_sequences(x_train,maxlen=max_length)
x_test=pad_sequences(x_test,maxlen=max_length)
print(x_train.shape,x_test.shape)

print(type(x_test))

model=Sequential([
    Embedding(vocab_size,64,input_length=max_length),
    SimpleRNN(64),
    Dense(1,activation='sigmoid')
])
model.compile(optimizer='adam',loss='binary_crossentropy',metrics=['accuracy'])
model.summary()

history=model.fit(x_train,y_train,epochs=3,batch_size=128,validation_split=0.2)
```

```
loss,acc=model.evaluate(x_test,y_test)
print("Test Accuracy:",acc)
pred=(model.predict(x_test)>0.5).astype(int)
print("Accuracy:",accuracy_score(y_test,pred))
print("Precision:",precision_score(y_test,pred))
print("Recall:",recall_score(y_test,pred))
print("F1:",f1_score(y_test,pred))
print("Confusion Matrix\n",confusion_matrix(y_test,pred))
print(classification_report(y_test,pred))

plt.plot(history.history['accuracy'],label='Train')
plt.plot(history.history['val_accuracy'],label='Validation')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid()

word_index = imdb.get_word_index()

review = input("Review: ").lower()
seq = [word_index.get(w.lower(), 2) + 3 for w in review.split()]
pred = model.predict(pad_sequences([seq], maxlen=max_length), verbose=0)[0][0]

print("Positive" if pred >= 0.5 else "Negative")
```

Output :

1. Dataset shapes

(25000, 200) (25000, 200)

2. Training logs

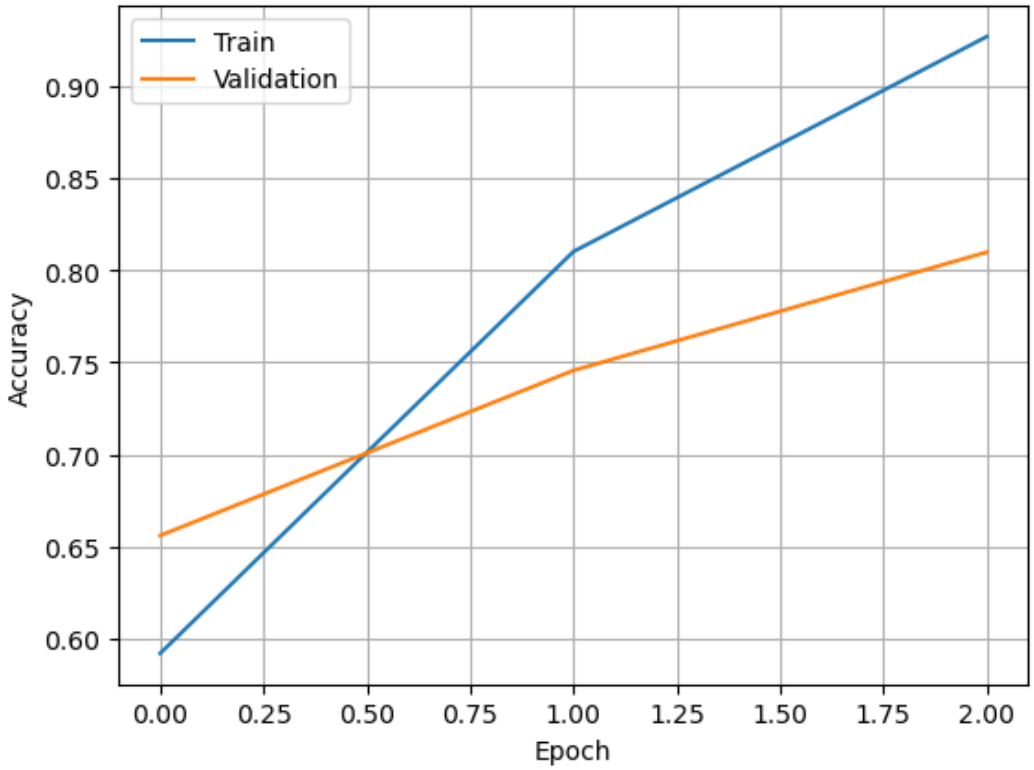
Epoch 1/3 accuracy: 0.5921 loss: 0.6539 val_accuracy: 0.6560 val_loss: 0.6105
Epoch 2/3 accuracy: 0.8101 loss: 0.4320 val_accuracy: 0.7456 val_loss: 0.5127
Epoch 3/3 accuracy: 0.9268 loss: 0.1982 val_accuracy: 0.8098 val_loss: 0.4740

3. Test metrics

Test Accuracy: 0.8143600225448608
Accuracy: 0.81436
Precision: 0.8004434589800443
Recall: 0.83752
F1: 0.8185621017240705
Confusion Matrix
[[9890 2610]
[2031 10469]]

	precision	recall	f1-score	support
0	0.83	0.79	0.81	12500
1	0.80	0.84	0.82	12500
accuracy			0.81	25000

4. Training vs validation accuracy



5. Sample prediction

Review: worse
Positive